UNIVERSITE DE FIANARANTSOA

FACULTE DE SCIENCES

MATHEMATIQUES ET APPLICATIONS

Parcours : Mathématiques Informatiques pour les Sciences Sociales

PROJET JAVA S8 EN MATHEMATIQUES INFORMATIQUES POUR LES SCIENCES SOCIALES

" DOCUMENTATION ET DEMARCHE POUR LE PROJET "

Numéro Carte d'étudiant	Nom des étudiants	Prénoms
M000714	ANDRIANARIVO	Samimanana Aimé
7982	TSIRIMANANDRAIBE	Tsilavinarivo Jean Thomas

1. Introduction

Ce document présente une analyse structurée et complète du projet SCOM exercice 3 (Système de Gestion Commerciale), réalisée à partir de l'ensemble des fichiers du projet (backend Java/Spring Boot, frontend React/Vite, scripts SQL) ainsi que du Modèle Conceptuel de Données (MCD) fourni par l'analyse SI. Le contenu de ce document repose exclusivement sur les éléments réellement présents dans le projet analysé.

Le document suit une progression chronologique : présentation générale, conception de la base de données, développement du backend, développement du frontend, architecture globale.

2. Présentation du projet SCOM

2.1 Objectif général

SCOM est une application web de gestion commerciale, conçue pour fonctionner entièrement en local (sans dépendance à un service Cloud, à Firebase ou à Auth0). Elle permet de gérer les vendeurs, les rayons (départements commerciaux), les bons de commande, les commissions, les paiements et les comptes utilisateurs, avec un tableau de bord et des rapports statistiques.

2.2 Problématique traitée

Le projet répond au besoin de suivre l'activité commerciale d'une entreprise organisée en rayons : affecter des vendeurs à des rayons, enregistrer les bons de commande qu'ils émettent, calculer les commissions associées, suivre les paiements reçus, et obtenir une vision synthétique (chiffres clés, classements, répartitions) de cette activité.

2.3 Acteurs identifiés

Le projet identifie, via la table et l'entité UTILISATEUR, trois rôles applicatifs utilisés dans le jeu de données de démonstration :

- ADMIN — compte administrateur (démonstration : admin / admin123)
- MANAGER — compte manager des ventes (démonstration : manager / manager123)
- OPERATEUR — compte opérateur de saisie (démonstration : operateur / operateur123)

2.4 Fonctionnement général

L'utilisateur se connecte via un formulaire de login (React), qui appelle l'API d'authentification du backend Hibernate. Une fois connecté, il accède à un tableau de bord et à des écrans de gestion (CRUD) pour chaque entité métier, ainsi qu'à une page de rapports/statistiques alimentée par un second backend, dédié aux requêtes analytiques.

2.5 Architecture globale et technologies

Le projet est structuré en quatre parties distinctes comme ci-dessous :

- backend-hibernate — API Spring Boot dédiée aux opérations CRUD (port 8081)

- backend-jpa — API Spring Boot dédiée aux statistiques et requêtes avancées Spring Data JPA (port 8082)
- frontend — application React (Vite) consommant les deux backends (port 5173)
- database — script SQL de création et de peuplement de la base MySQL (db_scom_database)

3. Analyse des besoins

À partir des entités et des relations réellement modélisées, les besoins métier suivants ont été identifiés :

- Organiser l'entreprise en rayons (départements) et affecter un ou plusieurs vendeurs à un ou plusieurs rayons.
- Enregistrer les bons de commande émis par chaque vendeur (numéro, date, montant, statut).
- Calculer et suivre les commissions perçues par les vendeurs sur leurs ventes.
- Enregistrer les paiements reçus, et associer un ou plusieurs paiements à un ou plusieurs bons de commande (règlement partiel ou groupé).
- Gérer des comptes utilisateurs internes pour l'authentification à l'application.
- Disposer d'un tableau de bord et de rapports (classements de vendeurs, statistiques par rayon, évolution mensuelle) pour le pilotage de l'activité.

4. Conception de la base de données

Cette section suit la progression demandée : MCD, puis MLD, puis structure physique réelle de la base MySQL.

4.1 MCD — Modèle Conceptuel de Données

Le MCD fourni (capture ci-dessous) modélise six entités et cinq associations :

Entités du MCD

Entité	Rôle
VENDEUR	Représente un vendeur (identité, coordonnées).
RAYON	Représente un rayon / département commercial.
COMMISSION	Représente une commission calculée pour un vendeur.
BON_COMMANDE	Représente un bon de commande émis par un vendeur.
PAIEMENT	Représente un règlement (encaissement).
UTILISATEUR	Représente un compte d'accès à l'application (authentification).

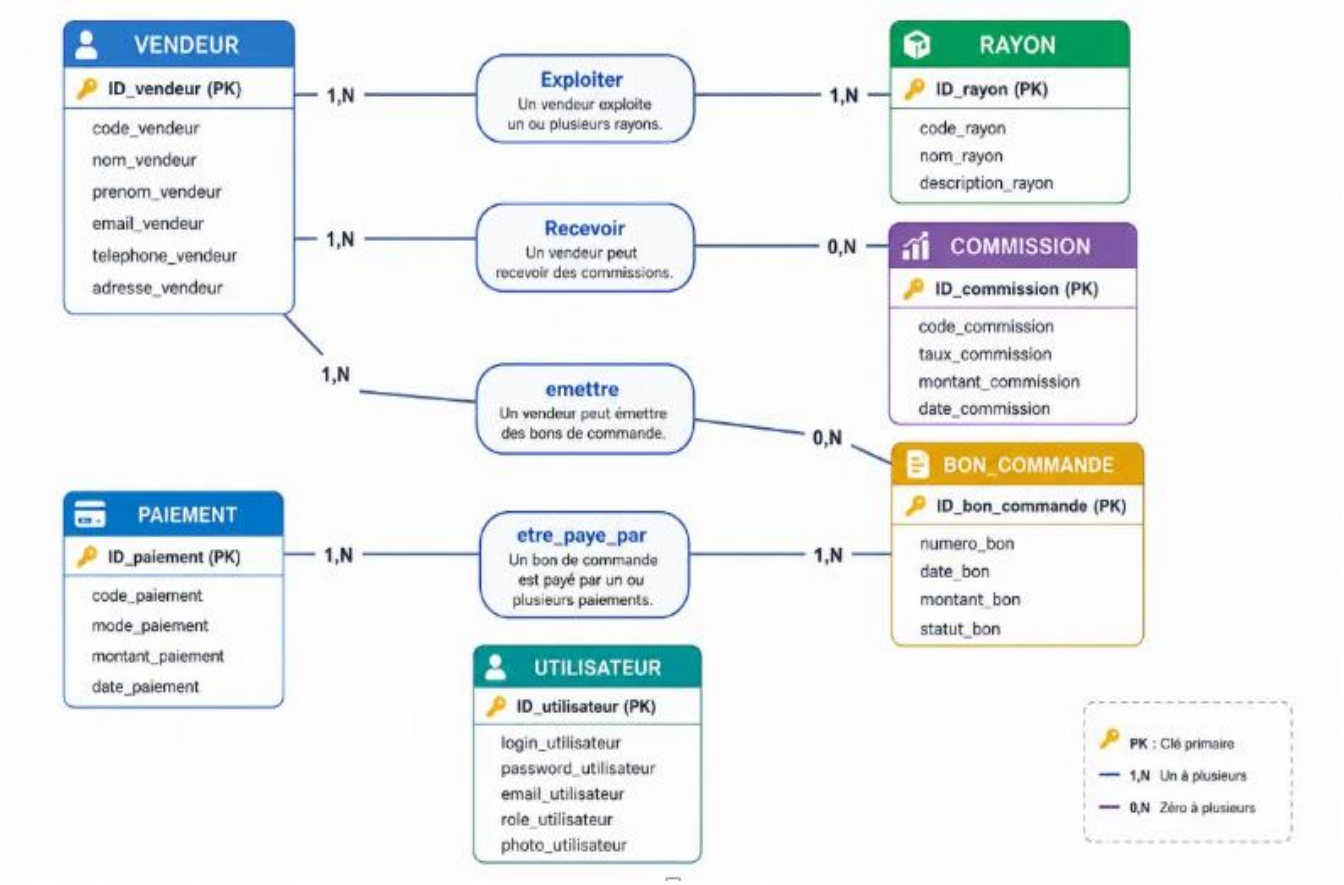
Associations et cardinalités du MCD

Association	Entités liées	Cardinalités	Signification
Exploiter	VENDEUR — RAYON	1,N — 1,N	Un vendeur exploite un ou plusieurs rayons ; un rayon est exploité par un ou plusieurs vendeurs (association many-to-many).
Recevoir	VENDEUR — COMMISSION	1,N — 0,N	Un vendeur peut recevoir des commissions.
Emettre	VENDEUR — BON_COMMANDE	1,N — 0,N	Un vendeur peut émettre des bons de commande.
Etre_paye_par	BON_COMMANDE — PAIEMENT	1,N — 1,N	Un bon de commande est payé par un ou plusieurs paiements (association many-to-many).
(implicite)	UTILISATEUR	—	Entité autonome, sans association représentée dans le MCD fourni.

Attributs des entités (tels que présents sur le MCD)

Entité	Attributs (MCD)
VENDEUR	ID_vendeur (PK), code_vendeur, nom_vendeur, prenom_vendeur, email_vendeur, telephone_vendeur, adresse_vendeur
RAYON	ID_rayon (PK), code_rayon, nom_rayon, description_rayon
COMMISSION	ID_commission (PK), code_commission, taux_commission, montant_commission, date_commission
BON_COMMANDE	ID_bon_commande (PK), numero_bon, date_bon, montant_bon, statut_bon
PAIEMENT	ID_paiement (PK), code_paiement, mode_paiement, montant_paiement, date_paiement
UTILISATEUR	ID_utilisateur (PK), login_utilisateur, password_utilisateur, email_utilisateur, role_utilisateur, photo_utilisateur

SCHEMA DU MODELE CONCEPTUEL DES DONNEES



4.2 MLD — Modèle Logique de Données

La transformation du MCD vers le modèle relationnel donne les tables suivantes, conformément au script SQL fourni (database/db_scom_database.sql) :

- RAYON (ID_rayon, code_rayon, nom_rayon, description_rayon)
- VENDEUR (ID_vendeur, code_vendeur, nom_vendeur, prenom_vendeur, email_vendeur, telephone_vendeur, adresse_vendeur)
- BON_COMMANDE (ID_bon_commande, numero_bon, date_bon, montant_bon, statut_bon, #ID_vendeur)
- COMMISSION (ID_commission, code_commission, taux_commission, montant_commission, date_commission, #ID_vendeur)
- PAIEMENT (ID_paiement, code_paiement, mode_paiement, montant_paiement, date_paiement)
- UTILISATEUR (ID_utilisateur, login_utilisateur, password_utilisateur, nom utilisateur, email_utilisateur, role_utilisateur)
- EXPLOITER (#ID_vendeur, #ID_rayon) — table de jonction pour l'association many-to-many VENDEUR ↔ RAYON
- ETRE_PAYE_PAR (#ID_bon_commande, #ID_paiement) — table de jonction pour l'association many-to-many BON_COMMANDE ↔ PAIEMENT

Les associations Recevoir et Emettre (portant des cardinalités 1,N côté VENDEUR) sont matérialisées par une clé étrangère ID_vendeur directement dans les tables COMMISSION et BON_COMMANDE, plutôt que par une table de jonction séparée — ce qui est cohérent avec des associations 1,N simples.

4.3 Structure physique (MySQL réelle)

La base est créée par le script database/db_scom_database.sql, en jeu de caractères utf8mb4 / collation utf8mb4_unicode_ci, moteur InnoDB pour toutes les tables. Le script crée 8 tables et insère un jeu de données de démonstration (4 rayons, 5 vendeurs, 8 bons de commande, 7 commissions, 5 paiements, 3 utilisateurs).

Table RAYON — départements commerciaux

Colonne	Type	Contraintes	Null	Défaut	Remarque
ID_rayon	BIGINT	PK	NOT NULL	—	Identifiant du rayon
code_rayon	VARCHAR(50)	UNIQUE (uk_rayon_code)	NOT NULL	—	Code métier du rayon
nom_rayon	VARCHAR(100)	—	NOT NULL	—	Nom du rayon
description_rayon	VARCHAR(500)	—	NULL	—	Description libre

Table VENDEUR — vendeurs

Colonne	Type	Contraintes	Null	Défaut	Remarque
ID_vendeur	BIGINT	PK	NOT NULL	—	Identifiant du vendeur
code_vendeur	VARCHAR(50)	UNIQUE (uk_vendeur_code)	NOT NULL	—	Code métier

Colonne	Type	Contraintes	Null	Défaut	Remarque
nom_vendeur	VARCHAR(100)	—	NOT NULL	—	
prenom_vendeur	VARCHAR(100)	—	NULL	—	
email_vendeur	VARCHAR(150)	—	NULL	—	
telephone_vendeur	VARCHAR(30)	—	NULL	—	
adresse_vendeur	VARCHAR(255)	—	NULL	—	

Table BON_COMMANDE — bons de commande

Colonne	Type	Contraintes	Null	Défaut	Remarque
ID_bon_commande	BIGINT	PK	NOT NULL	—	Identifiant du bon
numero_bon	VARCHAR(50)	UNIQUE (uk_bon_numero)	NOT NULL	—	Numéro affiché (ex. BC-2025-001)
date_bon	DATE	—	NULL	—	
montant_bon	DECIMAL(15,2)	—	NOT NULL	0	Montant du bon
statut_bon	VARCHAR(30)	—	NULL	—	Ex. PAYE / EN_COURS / ANNULE
ID_vendeur	BIGINT	FK → VENDEUR (fk_bon_vendeur), ON DELETE SET NULL	NULL	—	Vendeur émetteur (index idx_bon_vendeur)

Table COMMISSION — commissions

Colonne	Type	Contraintes	Null	Défaut	Remarque
ID_commission	BIGINT	PK	NOT NULL	—	
code_commission	VARCHAR(50)	UNIQUE (uk_commission_code)	NOT NULL	—	
taux_commission	DECIMAL(6,2)	—	NOT NULL	0	Taux en %
montant_commission	DECIMAL(15,2)	—	NOT NULL	0	
date_commission	DATE	—	NULL	—	
ID_vendeur	BIGINT	FK → VENDEUR (fk_commission_vendeur), ON DELETE SET NULL	NULL	—	Index idx_commission_vendeur

Table PAIEMENT — paiements

Colonne	Type	Contraintes	Null	Défaut	Remarque
ID_paiement	BIGINT	PK	NOT NULL	—	
code_paiement	VARCHAR(50)	UNIQUE (uk_paiement_code)	NOT NULL	—	
mode_paiement	VARCHAR(50)	—	NOT NULL	—	Ex. ESPECES / CB / VIREMENT

Colonne	Type	Contraintes	Null	Défaut	Remarque
montant_paiement	DECIMAL(15,2)	—	NOT NULL	0	
date_paiement	DATE	—	NULL	—	

Table UTILISATEUR — comptes applicatifs

Colonne	Type	Contraintes	Null	Défaut	Remarque
ID_utilisateur	BIGINT	PK	NOT NULL	—	
login_utilisateur	VARCHAR(100)	UNIQUE (uk_utilisateur_login)	NOT NULL	—	
password_utilisateur	VARCHAR(255)	—	NOT NULL	—	Stocké en clair (voir §9 sécurité)
nom_utilisateur	VARCHAR(100)	—	NULL	—	Absent du MCD fourni
email_utilisateur	VARCHAR(150)	—	NULL	—	
role_utilisateur	VARCHAR(50)	—	NULL	—	ADMIN / MANAGER / OPERATEUR

Table EXPLOITER — association VENDEUR ↔ RAYON (n:n)

Colonne	Type	Contraintes	Null	Défaut	Remarque
ID_vendeur	BIGINT	PK composite, FK → VENDEUR, ON DELETE CASCADE	NOT NULL	—	
ID_rayon	BIGINT	PK composite, FK → RAYON, ON DELETE CASCADE	NOT NULL	—	

Table ETRE_PAYE_PAR — association BON_COMMANDE ↔ PAIEMENT (n:n)

Colonne	Type	Contraintes	Null	Défaut	Remarque
ID_bon_commande	BIGINT	PK composite, FK → BON_COMMANDE, ON DELETE CASCADE	NOT NULL	—	
ID_paiement	BIGINT	PK composite, FK → PAIEMENT, ON DELETE CASCADE	NOT NULL	—	

5. Développement du Backend

Le projet contient deux applications Spring Boot indépendantes, partageant la même base MySQL mais avec des responsabilités distinctes.

5.1 Création et configuration du projet backend

Élément	backend-hibernate (port 8081)	backend-jpa (port 8082)
Spring Boot	3.5.16 (parent spring-boot-starter-parent)	3.5.16
Java	25 (release 25)	25
Build	Maven (pom.xml)	Maven (pom.xml)
Dépendances clés	spring-boot-starter-web, spring-boot-starter-data-jpa, mysql-connector-j, spring-boot-starter-validation, lombok, springdoc-openapi 2.8.6	identiques + h2 (tests) + byte-buddy (version property)
Connexion MySQL	jdbc:mysql://localhost:3306/db_scom_database	identique, + connectionCollation=utf8mb4_unicode_ci
ddl-auto	none (le SQL fourni fait foi)	none
CORS	CorsConfig.java — autorise http://localhost:5173	CorsConfig.java — autorise http://localhost:5173
Swagger/OpenAPI	/swagger-ui.html via springdoc	/swagger-ui.html via springdoc

5.2 Entités JPA

Les deux backends déclarent chacun leur propre jeu d'entités (mêmes noms de classes/tables, packages différents : com.scom.hibernate.entity et com.scom.jpa.entity). Le détail ci-dessous correspond au backend Hibernate, qui porte l'ensemble des relations JPA.

Entité	Table	Clé primaire	Relations JPA
Vendeur	VENDEUR	idVendeur	@OneToMany → BonCommande (mappedBy vendeur, cascade ALL, orphanRemoval) ; @OneToMany → Commission (idem) ; @ManyToMany → Rayon via @JoinTable EXPLOITER
Rayon	RAYON	idRayon	@ManyToMany(mappedBy="rayons") → Vendeur
BonCommande	BON_COMMANDE	idBonCommande	@ManyToOne → Vendeur (@JoinColumn ID_vendeur) ; @ManyToMany → Paiement via @JoinTable ETRE_PAYE_PAR
Commission	COMMISSION	idCommission	@ManyToOne → Vendeur (@JoinColumn ID_vendeur)
Paiement	PAIEMENT	idPaiement	@ManyToMany(mappedBy="paiements") → BonCommande
Utilisateur	UTILISATEUR	idUtilisateur	Aucune relation (entité autonome) ; champ photoUtilisateur (columnDefinition TEXT) sans colonne SQL correspondante

Toutes les entités utilisent Lombok (@Getter, @Setter, @NoArgsConstructor, @AllArgsConstructor, @Builder) et excluent les collections des méthodes equals/hashCode/toString (@ToString.Exclude, @EqualsAndHashCode.Exclude) afin d'éviter les boucles infinies liées aux relations bidirectionnelles.

5.3 Repositories

Chaque entité possède un repository Spring Data JPA (interface étendant JpaRepository), fournissant les opérations CRUD standard. Le backend JPA ajoute des méthodes dérivées et des requêtes personnalisées, notamment dans VendeurRepository :

- findByNomVendeurContainingIgnoreCase(String) — méthode dérivée pour la recherche par nom
- existsByCodeVendeur(String) — vérification d'unicité
- search(String q) — requête JPQL @Query recherchant sur code, nom et prénom (LIKE insensible à la casse)
- classement() — requête JPQL @Query avec projection DTO (SELECT new ...VendeurClassement(...)), agrégats COUNT/SUM et sous-requête corrélée sur COMMISSION

Les repositories RayonRepository, BonCommandeRepository, CommissionRepository et PaiementRepository du backend JPA exposent également des méthodes d'agrégation (sumMontants(), statistiques(), ventesParMois(), commissionsParMois()) utilisées par les services de statistiques.

5.4 Services

Le backend Hibernate suit un modèle classique interface + implémentation pour chaque entité (VendeurService/VendeurServiceImpl, etc.), avec les opérations : findAll, findById, create, update, delete, ainsi qu'une méthode spécifique authenticate (login) et updatePhoto pour Utilisateur.

Le backend JPA expose deux services dédiés à l'analytique :

- DashboardService / DashboardServiceImpl — assemble en un seul objet DashboardStats : nombre de vendeurs, de rayons, de bons, d'utilisateurs, sommes des montants (bons, commissions, paiements), top 5 des vendeurs, statistiques par rayon et séries mensuelles (ventes, commissions).
- StatistiqueService / StatistiqueServiceImpl — classementVendeurs() et statistiquesRayons(), réutilisées telles quelles par les repositories.

5.5 Controllers et endpoints

Backend Hibernate (port 8081, base /api)

Controller	Endpoint	Méthode	Fonction
VendeurController	/vendeurs	GET	Lister tous les vendeurs
VendeurController	/vendeurs/{id}	GET	Détail d'un vendeur
VendeurController	/vendeurs	POST	Créer un vendeur
VendeurController	/vendeurs/{id}	PUT	Mettre à jour un vendeur
VendeurController	/vendeurs/{id}	DELETE	Supprimer un vendeur
RayonController	/rayons (+ /{id})	GET/POST/PUT/DELETE	CRUD des rayons
BonCommandeController	/bons-commandes (+ /{id})	GET/POST/PUT/DELETE	CRUD des bons de commande
CommissionController	/commissions (+ /{id})	GET/POST/PUT/DELETE	CRUD des commissions
PaiementController	/paiements (+ /{id})	GET/POST/PUT/DELETE	CRUD des paiements
UtilisateurController	/utilisateurs (+ /{id})	GET/POST/PUT/DELETE	CRUD des utilisateurs
UtilisateurController	/utilisateurs/login	POST	Authentification (login/password)

Controller	Endpoint	Méthode	Fonction
UtilisateurController	/utilisateurs/{id}/photo	PUT	Mise à jour de la photo (base64)
HealthController	/health	GET	Vérification de disponibilité du service

Backend JPA (port 8082, base /api)

Controller	Endpoint	Méthode	Fonction
DashboardController	/dashboard	GET	Statistiques globales (KPI) du tableau de bord
StatistiqueController	/statistiques/classement-vendeurs	GET	Classement des vendeurs par ventes/commissions
StatistiqueController	/statistiques/rayons	GET	Statistiques agrégées par rayon
VendeurQueryController	/vendeurs	GET	Liste brute des vendeurs (entité exposée directement)
VendeurQueryController	/vendeurs/page	GET	Pagination + tri (Pageable/Sort)
VendeurQueryController	/vendeurs/search?q=	GET	Recherche full-text (code/nom/prénom)
HealthController	/health	GET	Vérification de disponibilité du service

5.6 Gestion des erreurs et validation

Chaque backend centralise la gestion des erreurs via un @RestControllerAdvice (GlobalExceptionHandler), qui traite ResourceNotFoundException (404), les erreurs de validation Bean Validation (400) et, côté Hibernate, un ConflictException (409) pour les violations de contraintes d'unicité. Les DTO de requête (xxxRequest) portent des annotations telles que @NotBlank, @Email, @Size, @DecimalMin, @NotNull, @Positive.

6. Fonctionnalités du Backend

- Gestion complète (CRUD) des vendeurs, rayons, bons de commande, commissions, paiements et utilisateurs (backend Hibernate).
- Authentification locale par login/mot de passe (comparaison de chaînes, sans hachage).
- Mise à jour de la photo de profil d'un utilisateur (chaîne base64 stockée en base).
- Calcul d'agrégats et de statistiques : totaux globaux, classement des vendeurs, statistiques par rayon, séries mensuelles de ventes et de commissions (backend JPA).
- Recherche full-text sur les vendeurs (code, nom, prénom) et pagination/tri génériques (backend JPA).
- Vérification de disponibilité (endpoint /health) sur chacun des deux backends.

7. Développement du Frontend

7.1 Création du projet frontend

Le frontend est un projet React initialisé avec Vite (vite.config.js, index.html). Les dépendances principales déclarées dans frontend/package.json sont : react et react-dom (18.3.1), react-router-dom (6.28), axios (1.7.9), recharts (2.15) pour les graphiques, react-icons (5.4) pour les icônes, et Tailwind CSS (@tailwindcss/postcss) pour une partie du style, complété par une feuille styles/index.css dédiée.

7.2 Architecture frontend

- components/ — composants réutilisables : CrudTable (tableau générique), Modal, Toast, ProtectedRoute (garde de route authentifiée), et un sous-dossier layout/ (AppLayout, Sidebar, Topbar) et common/ (Avatar, CurrencyDisplay).
- pages/ — une page par écran fonctionnel (voir §7.3), plus pages/auth/Login.jsx.
- context/AuthContext.jsx — gestion de l'état d'authentification via React Context, avec persistance dans localStorage (clé scom_user).
- services/ — un fichier par domaine, chacun basé sur les deux clients Axios définis dans services/api.js.
- utils/ — fonctions utilitaires : format.js, formatCurrency.js (formatage en Ariary), resizeImage.js (redimensionnement d'image côté client, utilisé pour la photo de profil).
- Routing : react-router-dom (composant Routes/Route dans App.jsx), avec une route publique /login et un groupe de routes protégées enveloppées par ProtectedRoute + AppLayout.

7.3 Pages de l'application

Page	Route	Objectif / contenu
Login	/login	Formulaire d'authentification (login + mot de passe), pré-rempli avec le compte admin de démonstration.
Dashboard	/dashboard	KPI globaux, graphiques (barres, camembert, aires) via Recharts, alimentés par le backend JPA (/dashboard) et par la liste des bons de commande.
Vendeurs	/vendeurs	Liste des vendeurs (CRUD) via CrudTable.
VendeurDetail	/vendeurs/:id	Détail d'un vendeur.
Rayons	/rayons	Liste et gestion des rayons.
BonsCommandes	/bons-commandes	Liste et gestion des bons de commande.
BonCommandeDetail	/bons-commandes/:id	Détail d'un bon de commande.
Commissions	/commissions	Liste et gestion des commissions.
CommissionDetail	/commissions/:id	Détail d'une commission.
Paiements	/paiements	Liste et gestion des paiements.
Utilisateurs	/utilisateurs	Gestion des comptes utilisateurs.
Rapports	/rapports	Statistiques avancées : classement des vendeurs et statistiques par rayon, explicitement décrites comme provenant du « backend JPA · port 8082 », avec graphiques Recharts.
Parametres	/parametres	Paramètres de l'application / du compte.

7.4 Services frontend et communication avec l'API

Le fichier services/api.js déclare deux instances Axios distinctes — hibernateApi (baseURL http://localhost:8081/api) et jpaApi (baseURL http://localhost:8082/api) — chacune dotée d'un intercepteur de réponse qui uniformise les erreurs (status, message, details). Chaque fichier de service choisit l'un ou l'autre client selon la nature de l'opération :

Service	Client Axios	Endpoints appelés	Opération
vendeurService	hibernateApi	/vendeurs (+/{id})	CRUD
vendeurService	jpaApi	/vendeurs/search, /vendeurs/page, /statistiques/classement-vendeurs	Recherche, pagination, classement
rayonService	hibernateApi	/rayons (+/{id})	CRUD
rayonService	jpaApi	/statistiques/rayons	Statistiques
bonCommandeService	hibernateApi	/bons-commandes (+/{id})	CRUD
commissionService	hibernateApi	/commissions (+/{id})	CRUD
paiementService	hibernateApi	/paiements (+/{id})	CRUD
utilisateurService	hibernateApi	/utilisateurs, /utilisateurs/login, /utilisateurs/{id}/photo	CRUD, authentification, photo
dashboardService	jpaApi	/dashboard	KPI globaux

Le flux général de communication est le suivant : Utilisateur → Interface React (page) → Service (Axios) → API Spring Boot (Controller) → Service métier → Repository (Spring Data JPA / Hibernate) → MySQL, et inversement pour la lecture des données.

8. Fonctionnalités du Frontend

- Authentification (connexion, déconnexion, persistance de session locale).
- Tableau de bord avec indicateurs clés et graphiques (barres, camembert, aires).
- Gestion CRUD (affichage, ajout, modification, suppression) pour vendeurs, rayons, bons de commande, commissions, paiements et utilisateurs, via un composant de tableau générique (CrudTable).
- Pages de détail pour vendeur, bon de commande et commission.
- Rapports et statistiques (classement des vendeurs, statistiques par rayon).
- Gestion de la photo de profil utilisateur, avec redimensionnement côté client avant envoi.
- Notifications/alertes (composant Toast) et fenêtres modales (composant Modal).
- Navigation latérale (Sidebar) avec les rubriques Dashboard, Vendeurs, Rayons, Bons de commande, Commissions, Paiements, Utilisateurs, Rapports & Stats, Paramètres.

9. Interface utilisateur

L'interface est structurée en trois zones : une barre latérale de navigation (Sidebar), une barre supérieure contextuelle (Topbar, affichant notamment titre/sous-titre de page et avatar utilisateur), et une zone de contenu principale (AppLayout). Les tableaux de données utilisent un composant générique réutilisable (CrudTable). Les graphiques (Dashboard et Rapports) sont réalisés avec la bibliothèque Recharts (barres, camemberts, aires). Les formulaires utilisent des composants standards (inputs, labels) avec gestion d'état React. Le style repose sur une feuille CSS dédiée (styles/index.css) combinée à des classes utilitaires Tailwind. Aucun élément d'internationalisation ni de mode sombre n'a été identifié dans le code fourni.

10. Architecture globale du projet

La synthèse de l'architecture réellement mise en œuvre est la suivante :

Couche	Rôle
MySQL (db_scom_database)	Stockage persistant des 8 tables ; source de vérité du schéma (ddl-auto=none).
Backend Spring Boot / Hibernate (8081)	Couche métier CRUD : Controller → Service → Repository → Entity, avec DTO Request/Response et mappers manuels.
Backend Spring Boot / Spring Data JPA (8082)	Couche analytique : agrégats, classements, pagination, recherche, exposée séparément du CRUD.
API REST (JSON, /api/**)	Contrat d'échange entre les deux backends et le frontend, sur deux ports distincts.
Frontend React / Vite (5173)	Interface utilisateur, routage, appels Axios vers les deux backends, mise en forme et visualisation des données.
Utilisateur final	Consulte et alimente les données via le navigateur.

11. Étapes de réalisation du projet SCOM

Chronologie déduite de la structure et de la cohérence du projet (analyse des besoins → base de données → backend → frontend) :

- 1. Analyse des besoins et identification des acteurs (vendeurs, rayons, commandes, commissions, paiements, utilisateurs).
- 2. Conception du MCD (entités, associations, cardinalités).
- 3. Passage du MCD au MLD (tables, clés primaires et étrangères, tables de jonction EXPLOITER et ETRE_PAYE_PAR).
- 4. Création du script SQL de la base MySQL (db_scom_database.sql), incluant un jeu de données de démonstration.
- 5. Création du projet backend-hibernate (Spring Boot, Maven, Java 25).
- 6. Configuration de la connexion MySQL et de Hibernate (ddl-auto=none, dialecte MySQL).
- 7. Création des entités JPA et de leurs relations (@ManyToOne, @OneToMany, @ManyToMany).
- 8. Création des repositories, mappers, DTO (Request/Response), services et controllers CRUD, avec gestion centralisée des erreurs.
- 9. Mise en place de l'authentification locale et de la gestion de la photo de profil.

- 10. Création du second projet backend-jpa, dédié aux statistiques (Dashboard, classements, pagination, recherche full-text).
- 11. Création du projet frontend (React + Vite), structuration en pages/composants/services/contexte.
- 12. Mise en place du routing (react-router-dom) et de la protection des routes (ProtectedRoute).
- 13. Création des services Axios, avec deux clients dédiés (hibernateApi / jpaApi) et connexion aux deux backends.
- 14. Implémentation des écrans CRUD, du tableau de bord et des rapports/statistiques.
- 15. Mise en place de tests unitaires ciblés côté backend (VendeurServiceTest) et de tests de démarrage de contexte Spring (smoke tests).

12. Tests et validation

Les éléments de test réellement présents dans le projet sont limités mais identifiables :

- backend-hibernate : ScomHibernateApplicationTests — test de démarrage du contexte Spring (smoke test).
- backend-hibernate : VendeurServiceTest — test unitaire de VendeurServiceImpl avec un VendeurRepository mocké (Mockito), vérifiant findById (cas trouvé et cas absent levant ResourceNotFoundException).
- backend-jpa : ScomJpaApplicationTests — test de démarrage du contexte Spring.

Le README indique que ces tests s'appuient sur une base H2 en mémoire (mode compatibilité MySQL) afin de ne pas nécessiter une instance WAMP/MySQL active ; la dépendance h2 est effectivement présente (scope test) dans le pom.xml du backend-jpa. Aucun test end-to-end frontend/backend, aucun test de composant React, et aucun résultat d'exécution de test n'ont été trouvés dans le projet ; ce document n'invente donc aucun résultat de test.

13. Technologies utilisées

Technologie	Utilisation dans le projet
Java 25	Langage du backend (les deux modules)
Spring Boot 3.5.16	Framework backend (Web, Validation, Data JPA)
Spring Data JPA	Accès aux données, repositories, requêtes dérivées et @Query
Hibernate	ORM sous-jacent (dialecte MySQL), utilisé nommément par le module backend-hibernate
MySQL (utf8mb4)	Base de données relationnelle (db_scom_database), moteur InnoDB
Lombok	Génération de code (getters/setters/constructeurs/builder) dans les entités
springdoc-openapi 2.8.6	Documentation API interactive (Swagger UI) sur les deux backends
React 18.3	Bibliothèque d'interface du frontend
Vite 6	Outil de build et serveur de développement du frontend
React Router DOM 6.28	Routage côté client
Axios 1.7.9	Communication HTTP avec les deux backends
Recharts 2.15	Graphiques (Dashboard, Rapports)
React Icons 5.4	Iconographie de l'interface

Technologie	Utilisation dans le projet
Tailwind CSS	Classes utilitaires de style, combinées à un CSS dédié
H2 (scope test)	Base en mémoire pour les tests automatisés backend
JUnit 5 / Mockito / AssertJ	Tests unitaires backend (spring-boot-starter-test)

14. Vérification de cohérence et incohérences relevées

La comparaison MCD ↔ MLD ↔ tables MySQL ↔ entités JPA ↔ repositories ↔ services ↔ controllers ↔ services Axios ↔ pages React fait apparaître les points de divergence suivants, tous vérifiés dans le code et/ou le script SQL fournis :

Point de contrôle	Constat
Champ photo_utilisateur (MCD ↔ SQL)	Présent sur le MCD et dans l'entité JPA Utilisateur (columnDefinition TEXT) et exposé via l'endpoint PUT /utilisateurs/{id}/photo, mais absent de la table UTILISATEUR du script SQL fourni.
Champ nom_utilisateur (MCD ↔ SQL)	Présent dans la table SQL UTILISATEUR et dans l'entité JPA, mais absent du MCD fourni.
Exposition d'entités JPA brutes	VendeurQueryController (backend JPA) renvoie directement List<Vendeur> / Page<Vendeur> (entité JPA), alors que le backend Hibernate encapsule systématiquement ses réponses dans des DTO xxxResponse ; le README affirme pourtant que les entités ne sont jamais exposées directement.
Duplication des entités/DTO	Les entités (Vendeur, Rayon, BonCommande, Commission, Paiement, Utilisateur) et une partie des DTO sont dupliquées à l'identique entre les packages com.scom.hibernate et com.scom.jpa, les deux modules étant des applications Spring Boot totalement indépendantes pointant vers la même base.
Sous-projet Next.js/PostgreSQL	Le projet contient à sa racine un gabarit de projet Next.js + Drizzle + PostgreSQL (package.json « nextjs-postgresql-template », src/app, src/db/schema.ts) sans aucun rapport avec la stack SCOM (Spring Boot/MySQL/React) ; il n'a pas été intégré à l'analyse fonctionnelle du projet.
Sécurité de l'authentification	Le mot de passe est stocké en clair (password_utilisateur VARCHAR) et comparé par égalité simple dans le service d'authentification ; aucune dépendance Spring Security n'est déclarée dans les pom.xml. Le README le signale lui-même comme limitation académique.
Association Recevoir / Emettre (1,N côté MCD)	Cohérentes avec l'implémentation : simple clé étrangère ID_vendeur dans COMMISSION et BON_COMMANDE, sans table de jonction, ce qui correspond bien à des cardinalités 1,N (et non n:n).
Associations Exploiter et Etre_paye_par (n:n)	Cohérentes : matérialisées par les tables de jonction EXPLOITER et ETRE_PAYE_PAR, retrouvées à l'identique dans le SQL et mappées par des relations @ManyToMany + @JoinTable côté entités JPA.

15. Conclusion

Le projet SCOM implémente une architecture en couches classique (MySQL → Spring Boot/Hibernate ou JPA → API REST → React/Vite), avec une particularité notable : la séparation volontaire des responsabilités entre deux backends Spring Boot indépendants — l'un dédié aux opérations CRUD via Hibernate, l'autre aux statistiques et requêtes avancées via Spring Data JPA — tous deux consommés par un frontend unique grâce à deux clients Axios distincts. La conception de la base de données (MCD, MLD, structure physique) est globalement cohérente avec le code, à l'exception de deux écarts mineurs sur les champs de la table UTILISATEUR et d'une exposition ponctuelle d'entités JPA brutes côté API de recherche. Le projet reste positionné comme un livrable académique, ce qui explique l'absence de sécurisation avancée (mots de passe en clair, pas de Spring Security/JWT), explicitement assumée dans la documentation du projet.